

REPORT



수강과목	딥러닝 응용
작성일	2024.12.19
학과	전자공학과
학번	2022049
이름	최재훈
제출일자	2024.12.20

목차

- 1 주제
- 2 문제점
- 3 목표
- 4 데이터셋 수집 및 전처리
- 5 기본 모델 구성
- 6 기본 모델 훈련 및 결과
- 7 하이퍼 파라미터 튜닝 비교
- 8 하이퍼 파라미터 튜닝 모델 구성
- 9 하이퍼 파라미터 튜닝 모델 훈련
및 결과
- 10 최종 결론 및 평가

1. 주제 : 날씨데이터와 어획량 데이터를 이용한 단가 예측

2. 문제점

해안 지역에 있는 수산 시장에 가면 다양한 수산물들을 판매하고 있다. 많은 사람들이 해안지역에 가면 수산 시장을 이용하는데 이때 시세에 따라 달라지는 수산물 가격에 많은 혼란을 겪고 덤탱이까지 씌여지는 경우도 빈번히 발생하고 있다. 이번 프로젝트를 통해서 이런 문제를 해결하고 사람들이 무분별한 가격 측정에 당하지 않고 현명한 소비를 할 수 있도록 지향한다.

3. 목표

최근 수산물 단가가 변동이 심해짐에 따라 일반 소비자들이 상인들의 무분별한 가격 책정에 피해를 입는 사례가 증가하고 있다. 이를 해결하기 위해 본 프로젝트는 해안 날씨 데이터와 어획량 데이터를 활용하여 수산물의 단가를 예측하는 모델을 개발하고, 소비자들이 수산물의 적정 가격을 쉽게 예측할 수 있도록 도움을 주고자 한다. 이를 통해 합리적인 소비와 더불어 공정한 가격 책정 문화를 확립하는 데 기여하고자 한다.

4. 데이터셋 수집 및 전처리

해당 프로젝트를 진행하기 위해 csv 파일로 구성된 데이터를 수집하였다. 데이터는 2021년 12월 01일부터 2024년 12월 01일까지 총 3년의 데이터를 획득하고 진행하였다. 먼저 날씨데이터를 수집하였는데 이때 어획과 가장 관련있다고 판단되는 요소를 위주로 수집하였다. 날씨데이터에는 일자, 평균기압(hPa), 평균 상대습도(%), 평균 기온(°C), 평균 수온(°C), 평균 유의 파고(m)를 포함하고 있다. 다음으로 어획량 데이터를 수집하였는데 사람들이 많이 찾는 고등어, 갈치, 갈고등어, 갑오징어, 오징어 종류별로 획득을 하였다. 여기서 갈치와 고등어는 크기별로 상/중/하를 구분하여 데이터를 수집하였다. 마지막으로 단가 데이터를 수집하였는데 이 데이터도 어획량 데이터와 같이 어종별로 구분해서 획득하였고 갈치와 고등어는 크기별로 구분하여 획득하였다. 이렇게 획득한 데이터를 입력값으로 넣기 쉽게 간단한 전처리를 진행하였다. 먼저 데이터를 하나의 파일로 만들고 데이터가 없는 결측값을 제거 하였다. 그렇게 데이터를 준비하였고 학습과 검증 테스트를 진행하기 위해서 학습과 테스트를 9:1로 나누었고 학습 중 8:2을 검증데이터로 나누었다. 총 데이터는 862개이고 학습 : 검증 : 테스트는 540 : 232 : 90으로 구성되어 있다.

일자	평균기압(hPa)	평균 상대습도(%)	평균 기온(°C)	평균 수온(°C)	평균 유의 파고(m)
2021-12-02	1020.1	54	6.9	17	1
2021-12-03	1017.2	56	11.1	17.8	
2021-12-04	1025.3	50	9	17.6	0
2021-12-06	1025.2	73	13.9	16	0
2021-12-07	1026	69	13.3	15.3	
2021-12-08	1030.5	65	12.9	17.1	1
2021-12-10	1024.2	75	14.3	16.7	0
2021-12-11	1023.9	69	13.2	17	0
2021-12-12	1021.6	58	11.3	17.8	0
2021-12-13	1024.6	48	5.8	17.4	1
2021-12-14	1022.2	56	7.7	17.9	0
2021-12-15	1021.5	67	13.3	19.1	0
2021-12-16	1017.5	74	12.2	17.8	0
2021-12-17	1018.9	53	6.1	16.7	1
2021-12-18	1025.4	51	2.2	16.3	1
2021-12-20	1019.4	60	11.7	16.1	0
2021-12-21	1016.4	64	13.6	16.3	
2021-12-22	1020.8	44	12.1	15.9	0
2021-12-23	1022.8	42	11.5	15.5	0
2021-12-24	1018.9	60	10.6	15.1	

날씨데이터

일자	갈지(상) 어획량	갈지(중) 어획량	갈지(하) 어획량	고등어(상) 어획량	고등어(중) 어획량	고등어(하) 어획량	갈고등어 어획량	갑오징어 어획량	오징어A 어획량
2021-12-02	4410	468	5382	1368	11106	33318	260166	180	
2021-12-03	5364	0	6480	0	0	0	540	270	
2021-12-04	18	0	18	8244	17100	70524	14796	36	
2021-12-06	22770	2502	12438	3870	30834	110142	56358	1692	
2021-12-07	3456	1548	4086	3780	23094	74988	266220	1494	
2021-12-08	1836	1944	36	2682	16326	70668	392484	1206	
2021-12-09	1728	2412	18	1404	18756	40464	355446	576	
2021-12-10	3330	0	2736	3042	12006	44946	684114	324	
2021-12-11	1386	252	216	3258	15444	49014	947850	1836	
2021-12-12	18	0	54	2934	12528	79002	1519734	0	
2021-12-13	16902	2664	16416	1440	9432	60948	1226316	2394	
2021-12-14	3420	3240	2952	324	8388	60408	888276	648	
2021-12-15	0	0	0	1404	7272	60822	918342	1116	
2021-12-16	90	520	2260	684	14004	43200	941604	3690	
2021-12-17	12240	6372	11232	1476	19008	46350	1120026	954	
2021-12-18	0	36	18	666	3636	26820	807156	2682	
2021-12-19									
2021-12-20	5850	1476	7506	486	5868	43542	663270	9162	
2021-12-21	0	0	36	0	0	0	80040	324	
2021-12-22	0	18	36	0	0	0	0	3024	
2021-12-23	126	0	234	0	0	0	0	18	
2021-12-24	4122	1872	11862	1620	14346	58230	692442	1656	
2021-12-25	594	864	5184	1458	6876	58176	965922	3114	

어획량 데이터

일자	갈치(상) 단가	갈치(중) 단가	갈치(하) 단가	고등어(상) 단가	고등어(중) 단가	고등어(하) 단가	갈고등어 단가	갑오징어 단가	오징어A 단가
2021-12-02	10328	4863	1753	6886	5006	4159	1200	7667	5
2021-12-03	10773	0	1544	0	0	0	1462	8989	
2021-12-04	5556	0	1500	7154	5934	4633	2949	2222	
2021-12-06	7406	3599	2131	6634	5630	4541	2524	9647	8
2021-12-07	13157	4438	1883	7358	6037	4586	1343	10246	6
2021-12-08	7898	4424	1028	6726	5129	4386	1059	9227	5
2021-12-09	13999	3789	2222	6290	5070	4331	1110	10490	4
2021-12-10	11979	0	1833	5610	4973	4172	894	7269	
2021-12-11	13740	3869	1343	4614	4477	3651	831	9071	5
2021-12-12	11111	0	1537	5391	4386	3967	779	0	4
2021-12-13	11859	4922	1728	6842	4650	3777	775	7958	5
2021-12-14	11954	4000	1222	6605	4941	3992	775	8606	
2021-12-15	0	0	0	8298	5195	4098	774	7938	
2021-12-16	17778	7787	1501	10497	5030	4003	763	9579	5
2021-12-17	12133	4176	1179	7249	5032	4029	740	6426	4
2021-12-18	0	3194	1111	7288	6097	4409	684	7824	
2021-12-19									
2021-12-20	11874	3624	1228	7572	6960	4802	727	8123	5
2021-12-21	0	0	1111	0	0	0	761	9660	
2021-12-22	0	3889	1861	0	0	0	0	9428	6
2021-12-23	7381	0	1808	0	0	0	0	5556	5
2021-12-24	10250	4665	2279	9728	5798	4448	875	8091	5
2021-12-25	10631	5463	1308	8145	4992	3977	726	8544	4

단가 데이터

일자	평균기아(평균)	상대(평균)	기온(평균)	수온(평균)	유의 갈치(상)	0갈치(중)	0갈치(하)	0고등어(상)	고등어(중)	고등어(하)	갈고등어	갑오징어	1오징어A	0갈치(상)	0갈치(중)	0갈치(하)	0고등어(상)	고등어(중)	고등어(하)	갈고등어	1갑오징어	1오징어	
2021-12-02	1020.1	54	6.9	17	1.1	4410	468	5382	1368	11106	33318	260166	180	38	10328	4863	6886	5006	4159	1200	7667	5	
2021-12-03	1017.2	56	11.1	17.8	1	5364	0	6480	0	0	0	540	270	0	10773	0	1544	0	0	1462	8989		
2021-12-04	1025.3	50	9	17.6	0.8	18	0	18	8244	17100	70524	14796	36	0	5556	0	1500	7154	5934	4633	2949	2222	
2021-12-06	1025.2	73	13.9	16	0.6	22770	2502	12438	3870	30834	110142	56358	1692	674	7406	3599	2131	6634	5630	4541	2524	9647	8
2021-12-07	1026	69	13.3	15.3	1	3456	1548	4086	3780	23094	74988	266220	1494	1368	13157	4438	1883	7358	6037	4586	1343	10246	6
2021-12-08	1030.5	65	12.9	17.1	0.7	1836	1944	36	2682	16326	70668	392484	1206	100	7898	4424	1028	6726	5129	4386	1059	9227	5
2021-12-10	1024.2	75	14.3	16.7	0.7	3330	0	2736	3042	12006	44946	684114	324	0	11979	0	1833	5610	4973	4172	894	7269	
2021-12-11	1023.9	69	13.2	17	0.5	1386	252	216	3258	15444	49014	947850	1836	1460	13740	3869	1343	4614	4477	3651	831	9071	5
2021-12-12	1021.6	58	11.3	17.8	0.7	18	0	54	2934	12528	79002	1519734	0	1060	11111	0	1537	5391	4386	3967	779	0	4
2021-12-13	1024.6	48	5.8	17.4	1.2	16902	2664	16416	1440	9432	60948	1226316	2394	340	11859	4922	1728	6842	4650	3777	775	7958	5
2021-12-14	1022.2	56	7.7	17.9	0.8	3420	3240	2952	324	8388	60408	888276	648	0	11954	4000	1222	6605	4941	3992	775	8606	
2021-12-15	1021.5	67	13.3	19.1	0.6	0	0	0	1404	7272	60822	918342	1116	0	0	0	0	8298	5195	4098	774	7938	
2021-12-16	1017.5	74	12.2	17.8	0.5	90	520	2260	684	14004	43200	941604	3690	90	17778	7787	1501	10497	5030	4003	763	9579	5
2021-12-17	1018.9	53	6.1	16.7	1.3	12240	6372	11232	1476	19008	46350	1120026	954	162	12133	4176	1179	7249	5032	4029	740	6426	4
2021-12-18	1025.4	51	2.2	16.3	1.5	0	36	18	666	3636	26820	807156	2682	0	0	3194	1111	7288	6097	4409	684	7824	
2021-12-20	1019.4	60	11.7	16.1	0.9	5850	1476	7506	486	5868	43542	663270	9162	36	11874	3624	1228	7572	6960	4802	727	8123	5
2021-12-21	1016.4	64	13.6	16.3	1	0	0	36	0	0	0	80040	324	0	0	1111	0	0	0	0	761	9660	
2021-12-22	1020.8	44	12.1	15.9	0.7	0	18	36	0	0	0	0	3024	18	0	3889	1861	0	0	0	0	9428	6
2021-12-23	1022.8	42	11.5	15.5	0.9	126	0	234	0	0	0	0	18	90	7381	0	1808	0	0	0	0	5556	5
2021-12-24	1018.9	60	10.6	15.1	1	4122	1872	11862	1620	14346	58230	692442	1656	306	10250	4665	2279	9728	5798	4448	875	8091	5
2021-12-25	1025.4	51	1.9	15	1.5	594	864	5184	1458	6876	58176	965922	3114	54	10631	5463	1308	8145	4992	3977	726	8544	4
2021-12-26	1030.7	50	-1.2	14.9	1.5	0	0	1548	15390	91926	1356990	0	0	0	0	0	0	6957	4856	3607	650	0	
2021-12-27	1029.6	53	1.8	15.3	1.5	6642	2862	8424	504	6678	49374	659082	3006	20	9763	4698	1380	6460	5177	3669	819	6256	20
2021-12-28	1027	58	7.2	15.1	1.1	0	0	918	4248	29664	372870	360	0	0	0	0	5980	4649	3325	856	8542		
2021-12-30	1022.3	51	5.5	14.4	1.2	3816	2880	5454	2124	16434	56718	1336368	1260	144	10338	3778	1419	7445	4910	3875	761	5906	54
2022-01-03	1024	44	7.4	15.5	0.7	2610	18	4932	720	5958	48960	1310784	450	270	8362	2889	1528	11000	6790	4216	798	8911	57
2022-01-04	1025.3	50	7.9	15.9	0.6	18	0	0	2016	12942	54648	881268	792	108	5556	0	0	8716	5355	3822	833	10423	58

데이터 통합을 거치고 간단한 전처리를 진행한 데이터

5. 기본 모델 구성

다음은 위에서 획득한 데이터를 불러와서 실행시킨 코드이다.

```
# 학습 데이터와 테스트 데이터 경로 입력
train_file_path = "C:/Users/user/Desktop/train.csv"
test_file_path = "C:/Users/user/Desktop/test.csv"

# CSV 파일 불러오기
train_data = pd.read_csv(train_file_path)
test_data = pd.read_csv(test_file_path)

# 랜덤시드 고정
random_seed = 12321
np.random.seed(random_seed)

# 학습 데이터 랜덤 셔플
train_data = train_data.sample(frac=1,
                                random_state=random_seed).reset_index(drop=True)

## 날짜 데이터에서 월 데이터 추가
#train_data['월'] = pd.to_datetime(train_data['일자']).dt.month
#test_data['월'] = pd.to_datetime(test_data['일자']).dt.month

# 날씨 데이터 컬럼
weather_columns = ["평균기압(hPa)", "평균 상대습도(%)", "평균
기온(°C)", "평균 수온(°C)", "평균 유의 파고(m)"]
catch_columns = [
    "갈치(상) 어획량", "갈치(중) 어획량", "갈치(하) 어획량",
    "고등어(상) 어획량", "고등어(중) 어획량", "고등어(하) 어획량",
    "갈고등어 어획량", "갑오징어 어획량", "오징어A 어획량"
]
price_columns = [
    "갈치(상) 단가", "갈치(중) 단가", "갈치(하) 단가",
    "고등어(상) 단가", "고등어(중) 단가", "고등어(하) 단가",
```

```

    "갈고등어 단가", "갑오징어 단가", "오징어A 단가"
]

# 데이터 정규화
scaler_X = MinMaxScaler()

# 어종별 학습
results = {}
loss_history = {}

for i, price_col in enumerate(price_columns):
    # 특정 수산물 어획량 선택
    catch_col = catch_columns[i]

    # 입력 데이터: 날씨 데이터 + 특정 어획량 + 월 데이터
    X_train = train_data[weather_columns + [catch_col]].copy()
    Y_train = train_data[price_col].copy()
    X_test = test_data[weather_columns + [catch_col]].copy()
    Y_test = test_data[price_col].copy()

    # 유효한 데이터(단가 > 0)만 사용
    valid_train_rows = Y_train > 0
    valid_test_rows = Y_test > 0
    if valid_train_rows.sum() < 10 or valid_test_rows.sum() < 10:
        print(f"{price_col} 학습/테스트 데이터가 부족하여 건너뛵니다.")
        continue

    X_train = X_train[valid_train_rows]
    Y_train = Y_train[valid_train_rows]
    X_test = X_test[valid_test_rows]
    Y_test = Y_test[valid_test_rows]

    # 데이터 정규화
    X_train_scaled = scaler_X.fit_transform(X_train)

```

```
X_test_scaled = scaler_X.transform(X_test)
```

어종별 단가를 예측하기 위해 학습데이터와 테스트 데이터를 불러와 데이터 전처리 및 정규화를 진행하였다. 먼저 학습 데이터와 테스트 데이터 경로를 설정하고 데이터를 불러온 뒤 데이터 셔플링을 진행하였다. 이후 분석에 필요한 주요 내용 3가지로 구분했으며 날씨 데이터 (기압, 습도, 기온, 수온, 유의 파고), 어종별 어획량 데이터(갈치, 고등어, 갑오징어 등), 단가 데이터(어종별 가격 정보)가 포함되었다. 데이터의 스케일 차이를 줄이기 위해 MinMaxScaler를 사용하여 입력 데이터를 정규화했으며, 학습 데이터로 정규화 모델을 학습한 뒤 동일한 방식을 테스트 데이터에도 적용하였다. 이후 각각의 어종별로 학습 및 테스트를 진행하였으며, 학습에 사용된 입력 변수는 날씨 데이터와 특정 어종의 어획량, 타깃 변수는 해당 어종의 단가로 설정하였다. 또한 단가가 0이거나 비어있는 경우는 제외하도록 하였다. 이 과정을 통해 어획량 및 날씨 데이터를 기반으로 어종별 단가를 예측하는 모델을 위한 데이터 준비를 끝냈다.

```
# 모델 정의
model = Sequential()
model.add(Dense(128, input_dim=X_train_scaled.shape[1],
activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(1, activation='linear'))
```

먼저 기본적인 성능을 확인하기 위해 간단한 인공신경망 모델을 만들었다. Sequential을 통해 layer들을 순차적으로 구성하였다. 입력 데이터의 차원에 맞춰 첫 번째 은닉층은 128개의 뉴런과 ReLU 활성화 함수를 사용하였다. 이후 두 번째 은닉층에서는 64개의 뉴런과 ReLU 활성화 함수를 추가하여 입력 데이터의 비선형성을 학습하도록 하였다. 마지막 출력층에서는 하나의 뉴런과 linear 활성화 함수를 사용하여 어종 단가를 예측하도록 구성하였다.

```
# 옵티마이저 및 모델 컴파일
optimizer = Adam(learning_rate=0.001)
model.compile(loss='MSE', optimizer=optimizer)

# 모델 학습
```



```

history = model.fit(
    X_train_scaled,
    Y_train.values,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

# 손실 기록 저장
loss_history[price_col] = {
    "loss": history.history['loss'],
    "val_loss": history.history['val_loss']
}

```

위 코드는 파라미터를 설정한 내용이다. 먼저 옵티마이저 Adam을 사용하였고 learning_rate를 0.001로 설정하였다. 모델 컴파일 과정에서 손실함수 MSE를 사용하였다. 모델 학습은 fit 함수를 통해 수행되며, 학습 데이터와 함께 100번의 에포크 동안 반복적으로 최적화를 진행하였다. 배치 크기는 32로 설정되어 학습데이터가 큰 단위로 나뉘어 빠르게 학습이 진행되도록 하였으며 여기서 학습 데이터 중 20%는 검증 데이터로 사용하게 하였다. 학습이 끝난 뒤 모델의 학습 손실값과 검증 손실값을 기록하여 저장하였다.

```

# 모델 평가
Y_pred = model.predict(X_test_scaled)

# 성능 지표 계산
mae = mean_absolute_error(Y_test, Y_pred)
mse = mean_squared_error(Y_test, Y_pred)
mape = np.mean(np.abs((Y_test.values - Y_pred.flatten()) /
Y_test.values)) * 100

```

위에서 진행한 학습을 기반으로 테스트를 진행하였다. 주어진 테스트 데이터에 대한 예측을 진행하였고 3가지 지표로 성능을 확인하였다. 평균

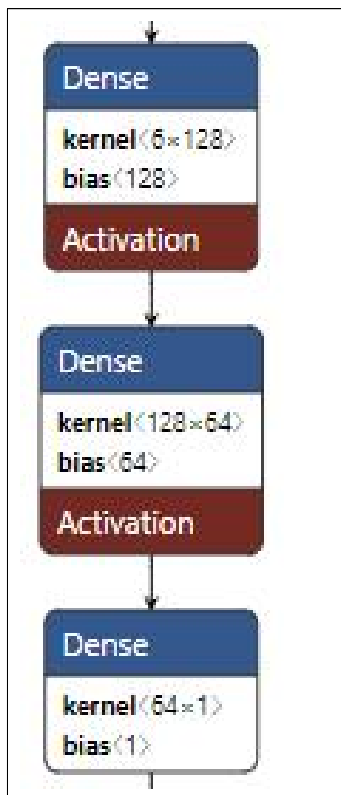
절대 오차 (MAE), 평균 제곱 오차(MSE), 평균 절대 백분율 오차(MAPE)를 사용하여 성능을 확인하였다.

평균 절대 오차(MAE): 실제값과 예측값 간의 절대 오차의 평균을 계산, 오차의 크기를 직관적으로 이해하는데 유용

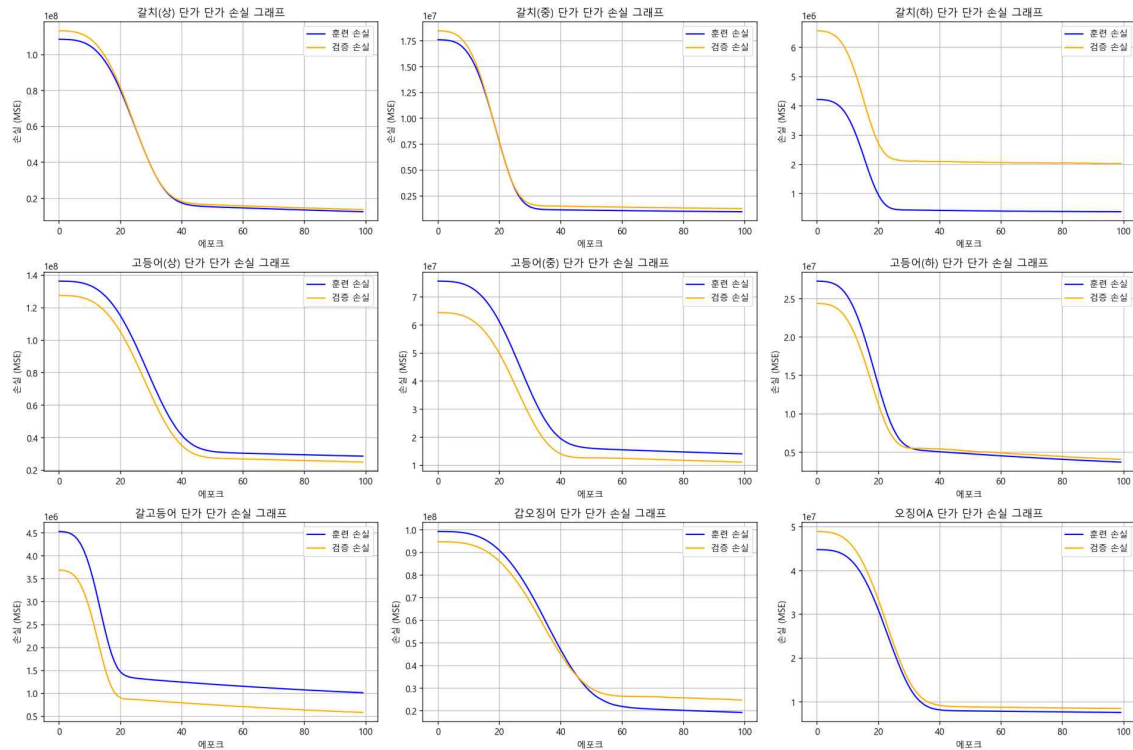
평균 제곱 오차(MSE): 실제값과 예측값의 차이를 제곱한 뒤 평균을 계산, 큰 오차에 더 민감하게 반응하여 모델의 전반적인 정확도를 평가

평균 절대 백분율 오차(MAPE): 실제값과 예측값이 실제값 대비 얼마나 비율적으로 차이가 나는지 표현

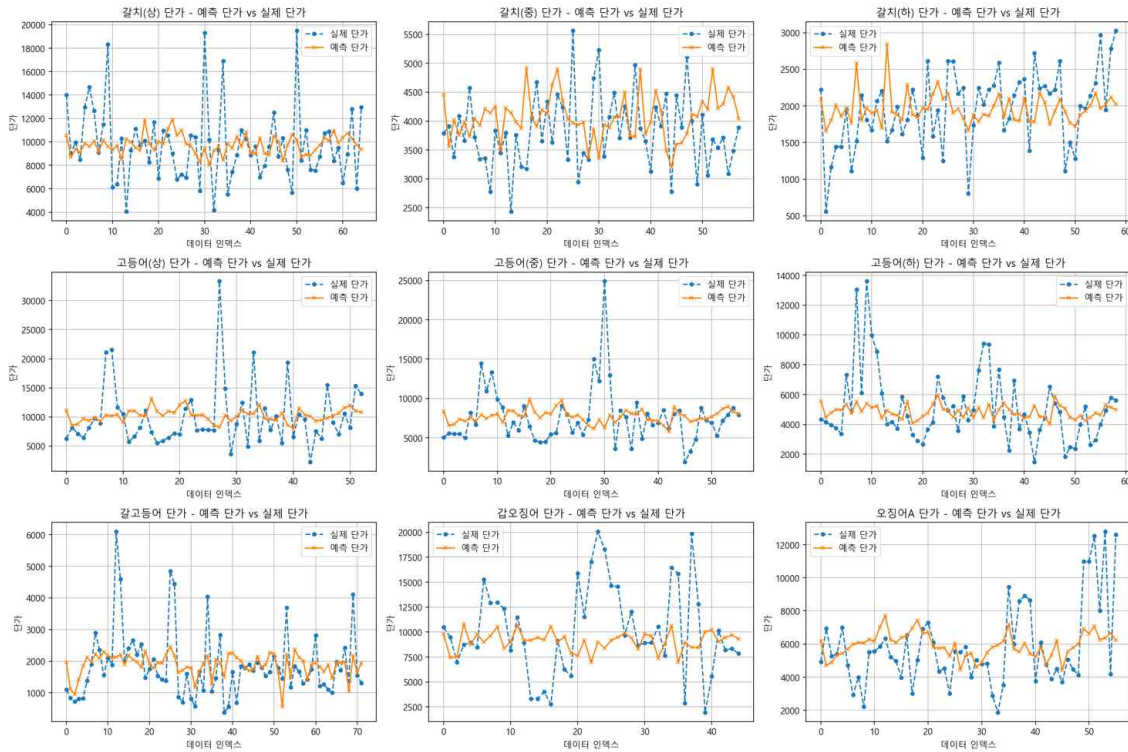
기본 모델 구성 그림



6.. 기본 모델 훈련 및 결과



위 사진은 기본 모델로 트레이닝을 하고난 어종별 손실 그래프이다. 그래프 전반적으로 에포크가 증가함에 따라 훈련 손실과 검증 손실이 급격히 감소한 뒤 특정 시점 이후 수렴하는 경향을 보이며, 이를 통해 모델이 데이터의 패턴을 학습하고 최적화되고 있음을 확인할 수 있었다. 초기 학습 단계에서는 손실 값이 빠르게 감소하여 모델의 학습 진행이 효과적으로 이루어지고 있음을 보여주며, 이후 에포크를 지나면서 손실 값이 점차 안정화되고 수렴하는 모습을 볼 수 있었다. 대부분의 어종에서 훈련 손실과 검증 손실 간의 차이가 크지 않아 모델이 과적합 없이 일반화가 잘 이루어졌음을 확인할 수 있었다. 다만, 일부 어종의 경우 검증 손실 값이 상대적으로 높게 나타나며, 이는 해당 어종의 단가와 어획량 및 날씨 데이터 간의 관계가 복잡하거나 데이터 품질의 문제일 가능성을 예상하게 한다. 전반적으로 그래프는 모델이 안정적으로 수렴하고 학습되었음을 보여주며, 특히 검증 손실이 낮고 안정적인 어종에 대해서는 신뢰할 만한 예측 결과를 제공할 수 있을 것으로 판단된다. 그러나 검증 손실이 높은 일부 어종에 대해서는 데이터 품질 개선이나 모델 구조 최적화를 통해 추가적인 개선 작업이 필요해 보인다.



위 사진은 테스트를 진행한 결과이다. 그래프에서 파란색은 실제 단가를 주황색은 예측된 단가를 시각화 하여 나타낸 것이다. 전체적으로 대부분의 어종에서 예측된 단가는 실제 단가의 추세를 대체로 잘 따라가고 있지만, 몇몇 경우에는 큰 편차가 발생하는 지점이 있었다. 특히 갈치(상)과 고등어(상) 같은 어종에서는 실제 단가의 변동 폭이 크지만, 이를 모두 반영하지 못하고 변동을 오치려 완화시킨 것을 확인할 수 있었다. 갈치(중), 갈치(하)와 같은 어종의 경우에도 일부 데이터 포인트에서 실제 단가와 예측 단가 사이의 차이가 크게 발생했으며, 데이터의 문제라고 판단 할 수 있을 것 같다. 그러나 전체적으로 예측 단가는 실제 단가의 추세를 비교적 잘 반영하고 있었으며, 기본적인 성능을 보여주는 동시에 하이퍼파라미터 튜닝을 통해 충분히 개선 여지가 보여진다고 판단할 수 있을 것 같다. 테스트 결과를 수치화 하면 다음과 같다.

MAPE	36.99%
MAE	2000.86

7. 하이퍼 파라미터 튜닝 비교

위에서 진행한 기본 모델의 성능을 개선하기 위해 하이퍼파라미터 튜닝을 진행하였다.

Batch_Size	32		16
Dropout(0.3)	O		X
Cov1D	O		X
Dense Layer 추가	X	32	32,16
Epoch	300		

위에서 나온대로 Batch_Size를 32나 16으로 변경하여 진행하였고 Dropout(0.3)도 추가해 보았다. dense층을 Conv1D로 변경해보기도 하였고 층을 늘려보기도 하였다. 마지막으로 Epoch도 300으로 증가시키고 진행하였다. 이렇게 총 24가지 경우의 수 파라미터 튜닝을 진행시킨 결과는 다음과 같다.

하이퍼파라미터	MAPE	MAE
Batch16_DropoutTrue_Dense_4Layers_results	29.27	1650.7
Batch32_DropoutTrue_Dense_4Layers_results	29.83	1685.35
Batch16_DropoutTrue_Dense_3Layers_results	30.64	1677.07
Batch32_DropoutFalse_Dense_4Layers_results	30.8	1707.01
Batch16_DropoutFalse_Dense_4Layers_results	31.42	1669.86
Batch32_DropoutTrue_Dense_3Layers_results	31.89	1749.81
Batch16_DropoutFalse_Dense_3Layers_results	31.94	1706.48
Batch16_DropoutTrue_Conv1D_4Layers_results	32.64	1764.85
Batch32_DropoutFalse_Dense_3Layers_results	32.91	1751.36
Batch16_DropoutFalse_Conv1D_4Layers_results	33.29	1754.13
Batch32_DropoutTrue_Conv1D_4Layers_results	33.46	1798.86
Batch32_DropoutTrue_Conv1D_3Layers_results	33.71	1821.44
Batch16_DropoutTrue_Conv1D_3Layers_results	33.72	1784.66
Batch16_DropoutTrue_Dense_2Layers_results	33.79	1815.96
Batch16_DropoutFalse_Conv1D_3Layers_results	33.86	1802.3
Batch16_DropoutTrue_Conv1D_2Layers_results	33.99	1817.52
Batch32_DropoutFalse_Conv1D_3Layers_results	34.04	1801.9
Batch32_DropoutFalse_Conv1D_4Layers_results	34.12	1794.83
Batch16_DropoutFalse_Dense_2Layers_results	34.25	1815.61
Batch16_DropoutFalse_Conv1D_2Layers_results	34.46	1833.92
Batch32_DropoutFalse_Conv1D_2Layers_results	34.51	1843.99
Batch32_DropoutTrue_Conv1D_2Layers_results	34.62	1837.49
Batch32_DropoutTrue_Dense_2Layers_results	34.87	1872.21
Batch32_DropoutFalse_Dense_2Layers_results	35.3	1875.68

위의 결과를 보았을 때 전체적으로 Conv1D를 사용하면 성능이 떨어지는 것을 확인할 수 있었다. Batch_Size도 작은 경우에 더 성능이 높은 것 또한 확인할 수 있었다. 층 또한 많을수록 더 좋은 성능을 보이고 Dropout도 있는게 더 좋은 효과를 보인 것 같다. 그중에 가장 잘 나온 것은 Batch_Size=16, Dropout(0.3) 적용, Dense 층을 사용하고 16, 32 뉴런의 층을 추가시킨 경우이다.

8. 하이퍼 파라미터 튜닝 모델 구성

```
# 모델 정의
model = Sequential()
model.add(Dense(128, input_dim=X_train_scaled.shape[1],
activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(16, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='linear'))
```

위에서 작성한 기본 코드를 베이스로 작성하였다. 128개의 뉴런을 가진 첫 번째 은닉층에서 입력 데이터를 처리하며, 두 번째와 세 번째 은닉층에서는 각각 64개와 32개의 뉴런을 사용하여 데이터를 점진적으로 압축하고 특징을 추출하도록 하였다. 세 번째 은닉층 이후에는 Dropout(0.3)을 적용하여 학습 중 일부 뉴런을 무작위로 비활성화함으로써 과적합을 효과적으로 방지하고 모델의 일반화 성능을 강화했다. 이후 16개의 뉴런을 가진 마지막 은닉층에서도 동일한 방식으로 Dropout이 적용되었으며, 최종적으로 하나의 뉴런과 linear 활성화 함수를 사용하는 출력층에서 단가 값을 예측하도록 하였다. 은닉층을 더 깊게 하고 Dropout을 통해 데이터의 복잡한 패턴을 학습할 수 있도록 설계하였다.

```
# 옵티마이저 및 모델 컴파일
optimizer = Adam(learning_rate=0.001)
model.compile(loss='MSE', optimizer=optimizer)
```

```

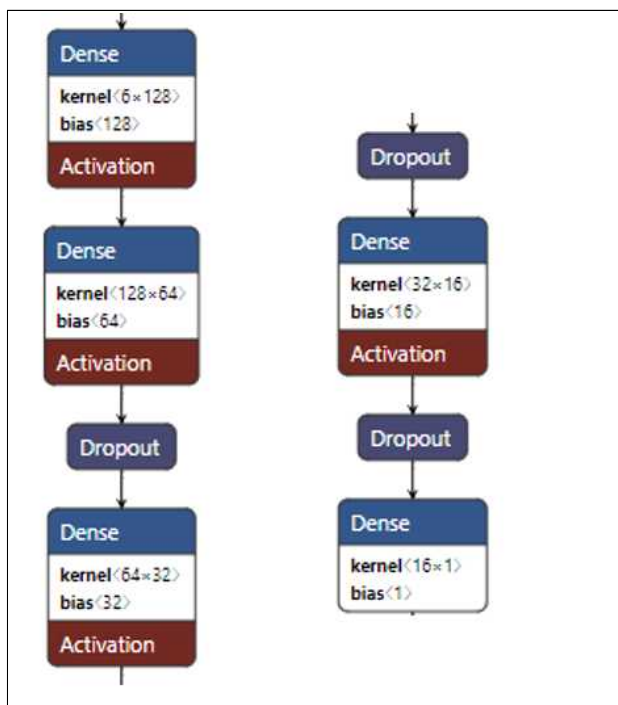
# 모델 학습
history = model.fit(
    X_train_scaled,
    Y_train.values,
    epochs=300,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

# 손실 기록 저장
loss_history[price_col] = {
    "loss": history.history['loss'],
    "val_loss": history.history['val_loss']
}

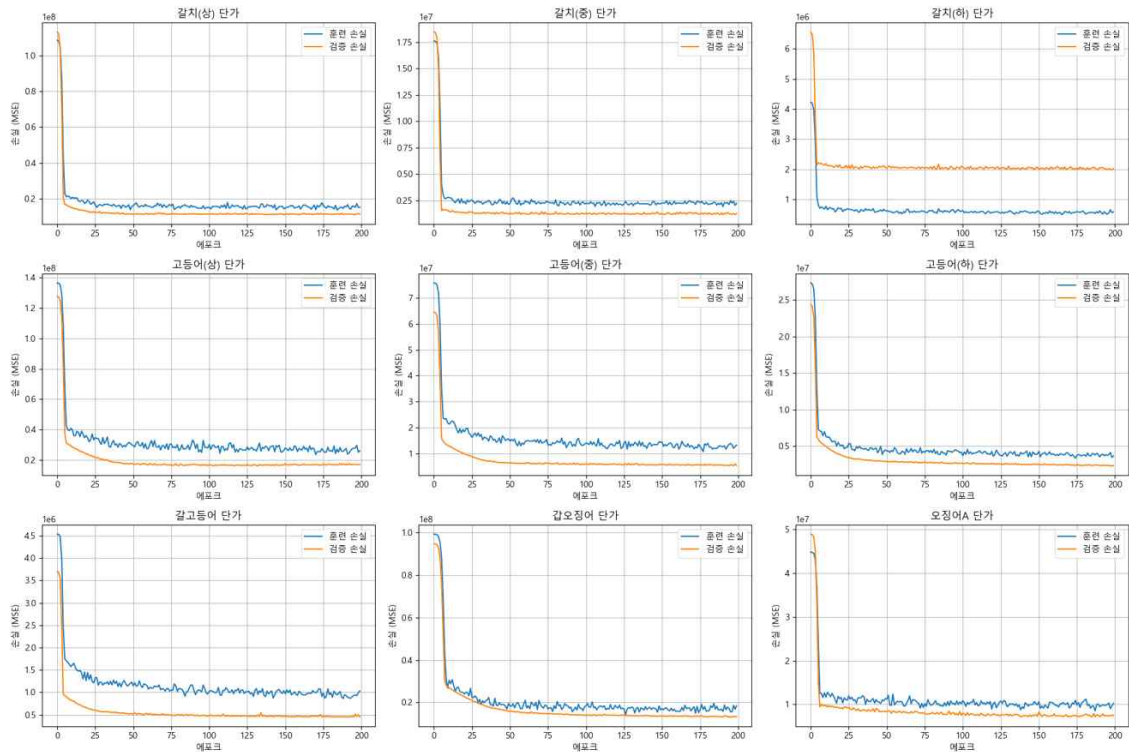
```

부족한 학습을 진행하기 위해 Epochs를 300으로 늘렸고 Batch_Size를 16으로 감소시켰다.

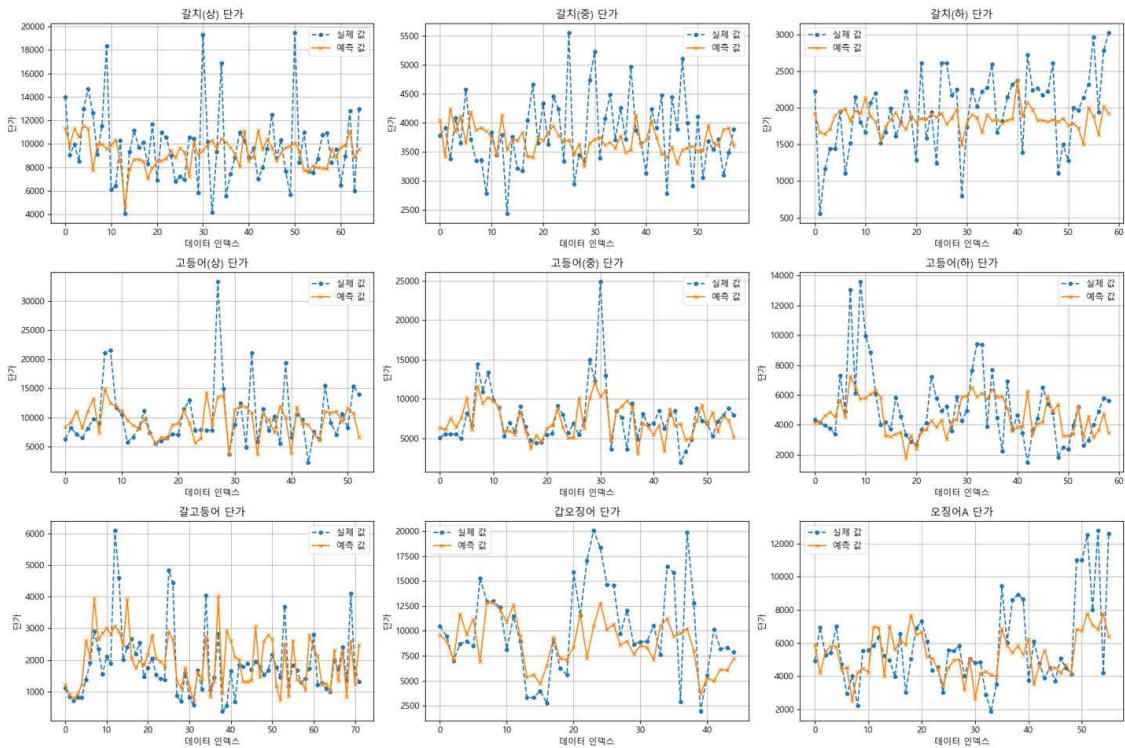
하이퍼파라미터 튜닝 모델 구성 그림



9. 하이퍼 파라미터 튜닝 모델 훈련 및 결과



위 사진은 하이퍼파라미터를 조정한 학습결과이다. 이전 그래프와 비교해 학습 손실과 검증 손실의 경향성을 더 세밀하게 관찰할 수 있었다. 기본 그래프와 같이 에포크가 진행됨에 따라 손실 값이 감소하는 공통적인 패턴을 보이지만, 두 번째 그래프에서는 학습의 안정성과 수렴 단계에서의 손실 변화가 더욱 뚜렷하게 나타낸 것을 확인할 수 있다. 하이퍼파라미터 조정과 같은 개선 작업이 효과적으로 작용한 것을 확인할 수 있었다. 특히 일부 어종에서는 손실 값이 기본 모델 그래프보다 전반적으로 낮아 모델의 예측 성능이 개선된 것을 확인할 수 있다. 하지만 오징어A와 같은 경우는 여전히 부족한 부분을 확인할 수 있었다.



위 그래프는 하이퍼파라미터 튜닝한 모델로 테스트를 돌린 결과이다. 기본 모델 테스트 결과 그래프와 비교했을 때 전반적으로 예측 성능이 개선된 모습을 볼 수 있었고 단가의 변동 패턴을 더 잘 반영하고 오차를 줄이는 것을 확인할 수 있었다. 이러한 결과 하이퍼파라미터 최적화 및 학습 과정 개선이 예측 성능 향상에 긍정적인 영향을 미친 것을 확인했다. 다만 일부 어종에서는 여전히 실제 값과 예측 값 간의 차이가 존재하므로 더욱 추가적인 연구가 필요해 보인다.

10. 최종 결론 및 평가

이번 프로젝트에서는 날씨와 어획량 데이터를 활용하여 어획량 단가를 예측하고 실제 값과 비교하는 작업을 진행하였다. 주요 결과로는, 날씨 조건에 따른 단가 흐름은 효과적으로 예측할 수 있었으나, 단가의 급등이나 급락 상황에서는 예측 정확도가 낮아지는 한계가 있었다. 하이퍼파라미터 튜닝을 통해 예측 오차 범위를 줄이는 데 성공했지만, 급격한 변동 상황은 데이터 부족이나 날씨 외의 추가 요인들이 영향을 미친 것으로 분석된다. 날씨 데이터만으로는 완전한 예측이 어렵다는 점에서 추가적인 데이터가 필요함을 확인하였다. 하이퍼파라미터 튜닝을 통해 일반적인 상황에서의 예측 오차를 줄일 수 있었으나, 비정상적인 단가 변동을 예측하는 데에는 한계가 있었습니다. 데이터 다양성과 정확성의 부족이라고 판단하게 되었다.

향후 계획으로는 데이터 다양성과 모델 성능 강화를 목표로 하고있다. 단가 변동에 영향을 줄 수 있는 외부 요인 데이터를 추가로 수집하고 LSTM 기반의 비정상 감지 기법을 도입할 예정이다.